



Algorithmen und Datenstrukturen

Leon Johann Brettin



Algorithmen und Datenstrukturen

14. Graphen



Graphen

- Netzwerke aus Knoten und Kanten
- vielfältiger Einsatz
 - Verbindungsnetzwerke: Bahnnetz, Flugverbindungen, Straßenkarten, ...
 - Verweise: WWW, Literaturverweise, Wikipedia, symbolische Links, ...
 - technische Modelle: Platinen-Layout, Computergraphik, ...
 - Software-Dokumentation
- Bäume und Listen sind spezielle Graphen



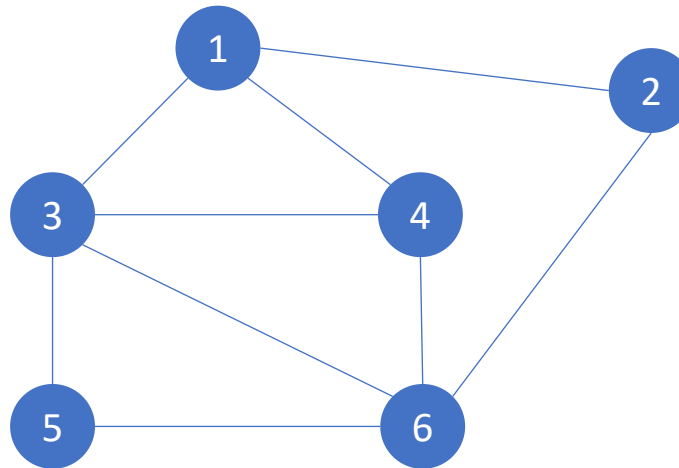
Definitionen

- Ein **ungerichteter Graph** $G = (V, E)$ besteht aus einer endlichen Menge V von **Knoten** (vertices, auch **Ecken**) und einer Menge E von zwei-elementigen Teilmengen von V , den **Kanten** (edges).
- Bei einer Kante $k = \{a, b\}$ mit $a, b \in V$ heißen a, b **Endpunkte** von k .
- a und b sind **benachbart** und mit k **inzident**.
 - Keine Pfeile
 - Keine Mehrfachkanten
 - Keine Schleifen



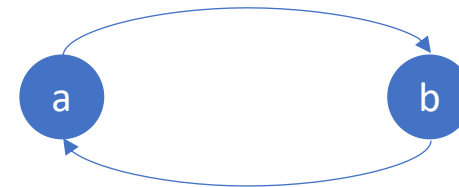
Beispiel für einen ungerichteten Graph G_u

- $G_u = (V_u, E_u)$
- $V_u = \{1, 2, 3, 4, 5, 6\}$
- $E_u = \{\{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 6\}, \{4, 6\}, \{3, 6\}, \{5, 6\}, \{3, 4\}, \{3, 5\}\}$



Definitionen

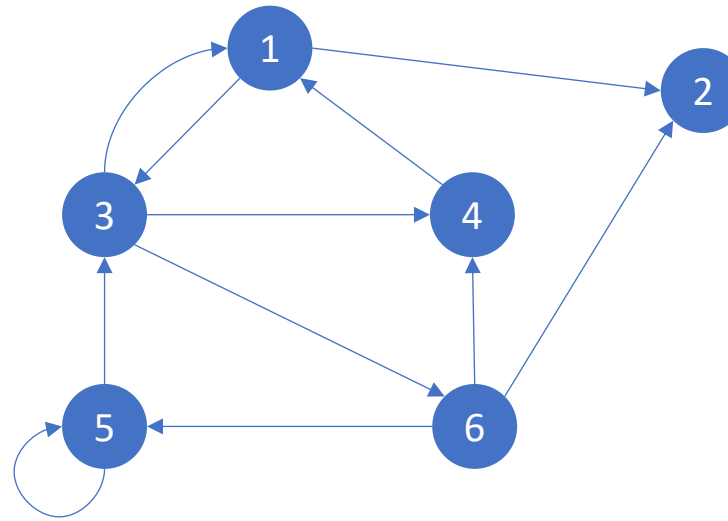
- Ein **gerichteter Graph** (Digraph) $G = (V, E)$ besteht aus einer endlichen Menge V von **Knoten** und einer Menge E von geordneten Paaren $(a, b) \in V \times V$ mit $a \neq b$, den gerichteten **Kanten**.
- Bei einer Kante $k = (a, b)$ heißt a Anfangspunkt, b **Endpunkt** von k .
- a und b sind **benachbart** und mit k **inzident**.
 - Pfeile
 - Keine Mehrfachkanten
 - Keine Schleifen
- E : (irreflexive) Relation in V
- Ungerichteter Graph $\rightsquigarrow$ gerichteter Graph: jede Kante durch zwei gerichtete Kanten ersetzen. Symmetrische Relation.
- Zusammenfassen von Kanten (a,b) und (b,a) in der grafischen Repräsentation ok.





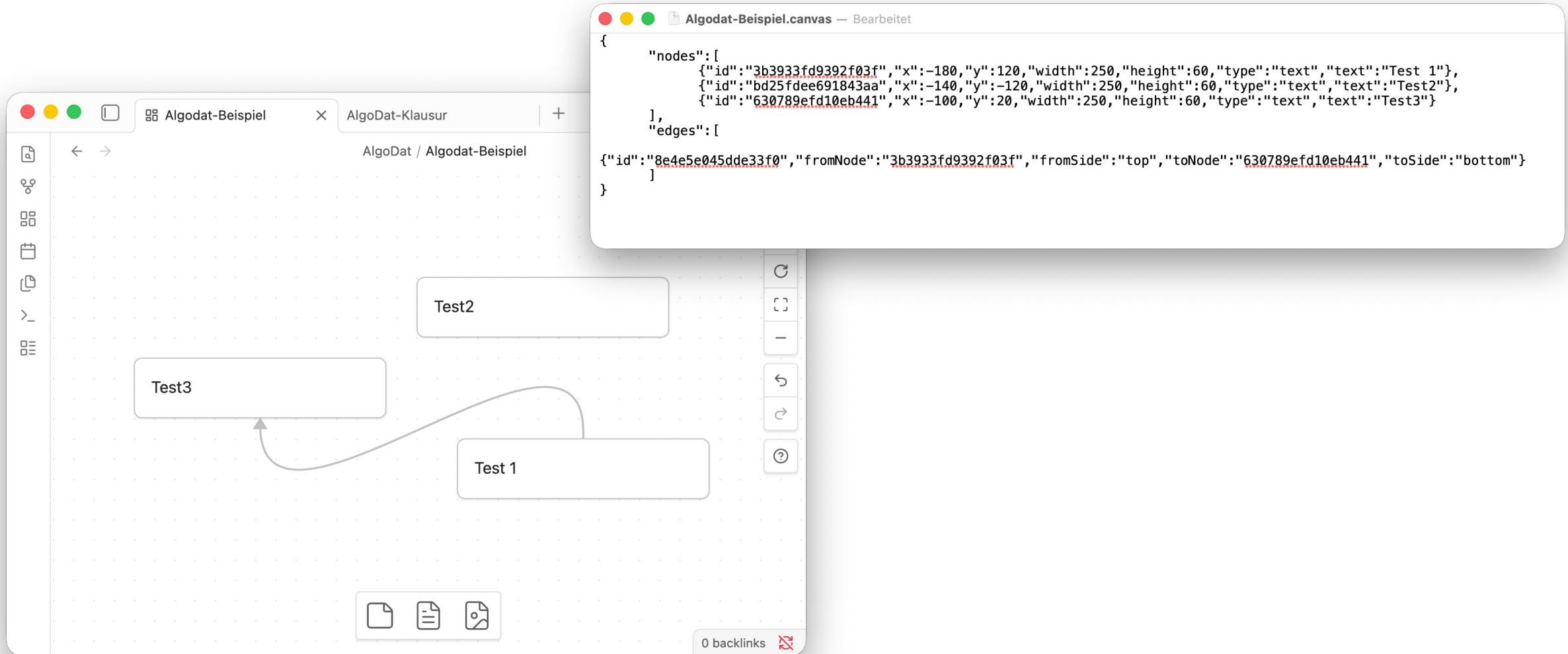
Beispiel für einen gerichteten Graph G_g

- $G_g = (V_g, E_g)$
- $V_g = \{1, 2, 3, 4, 5, 6\}$
- $E_g = \{(1, 2), (1, 3), (3, 1), (4, 1), (3, 4), (3, 6), (5, 3), (5, 5), (6, 5), (6, 2), (6, 4)\}$



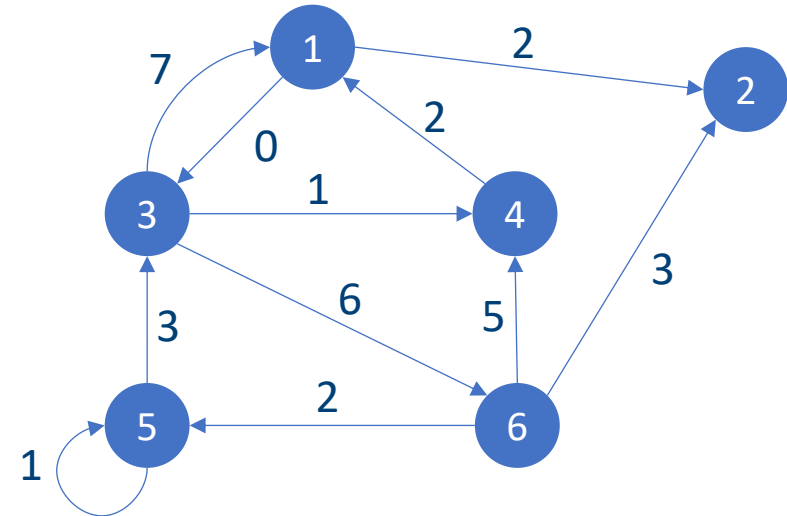


Graph Echtes Beispiel



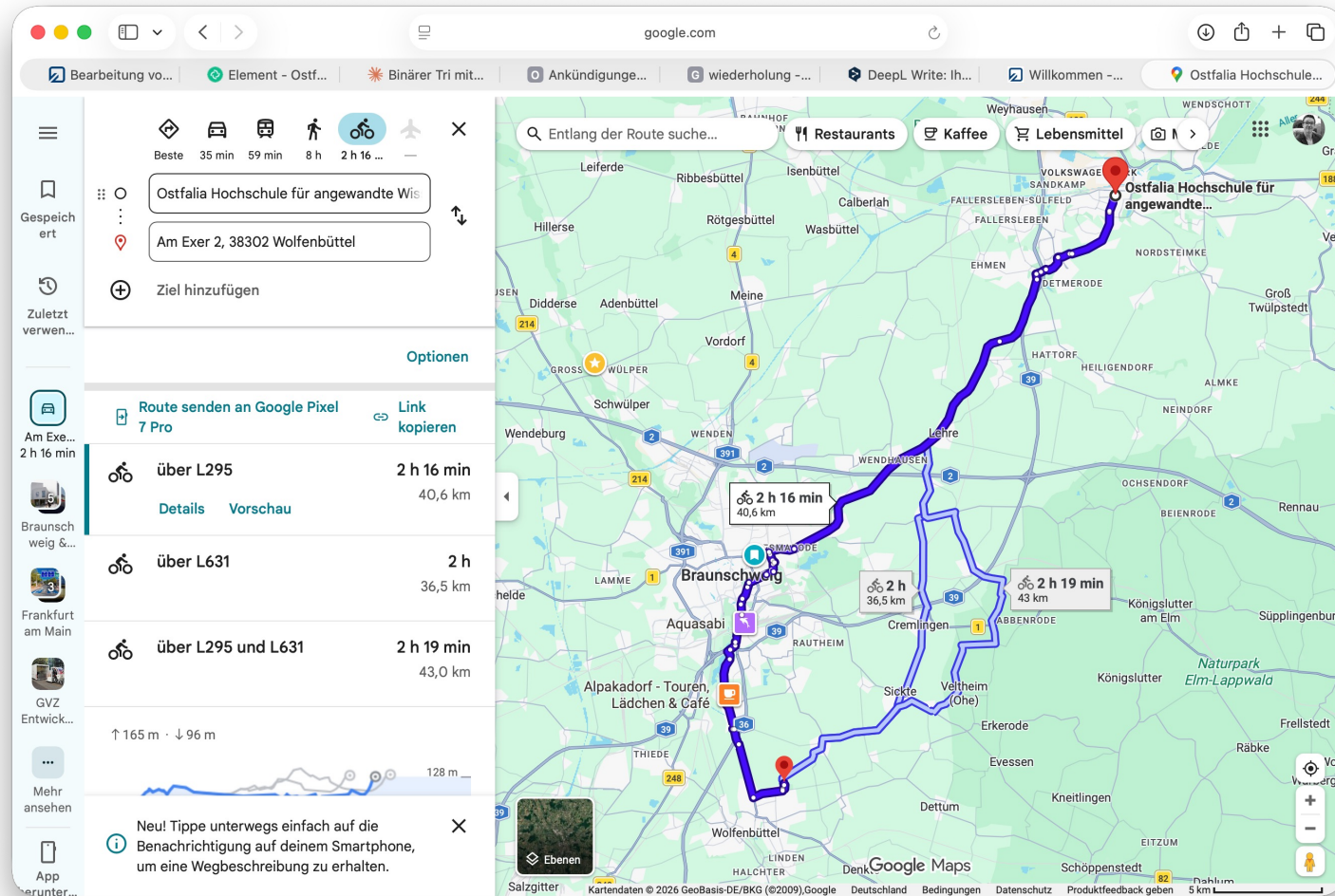
Gewichtete Graphen

- Graph + Kantengewichte γ , zum Beispiel natürliche Zahlen
- $G = (V, E, \gamma)$ mit $\gamma: E \rightarrow \mathbb{N}$
 - gerichtet oder ungerichtet
- Anwendungen:
Suche kürzeste Wege, maximalen Fluss, minimale Transportkosten, ...



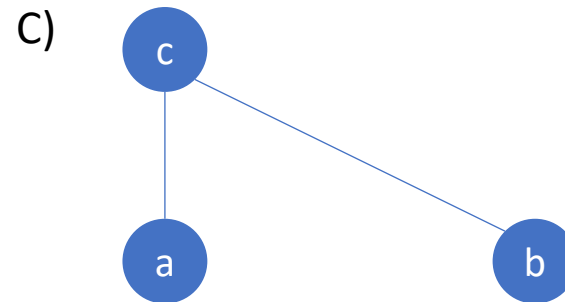
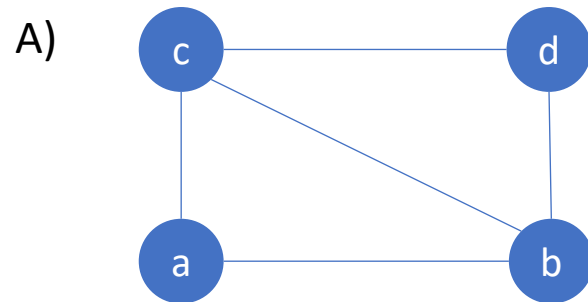


Gewichteter Graph „echtes“ Beispiel

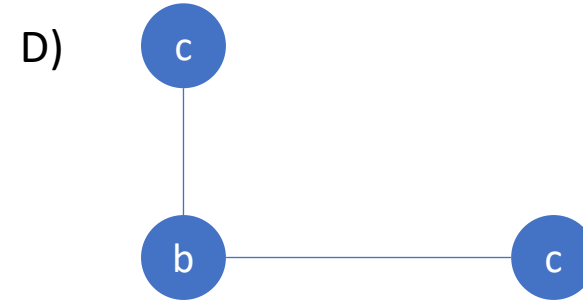
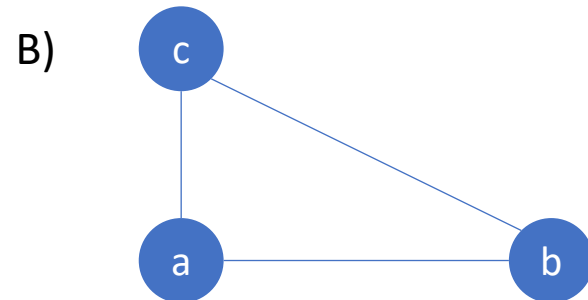


Übung

- Welcher Graph ist $(\{a,b,c\}, \{\{a,c\}, \{a,b\}\})$?

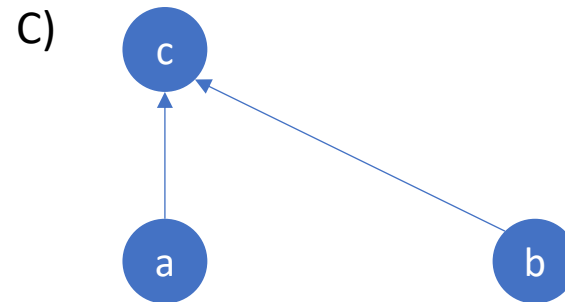
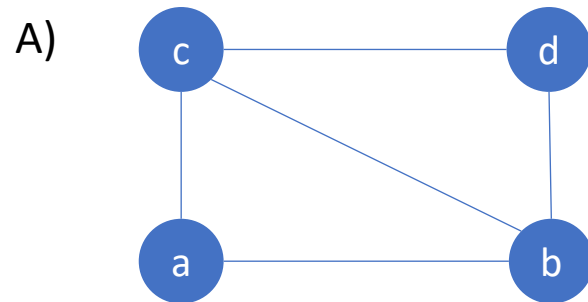


E)

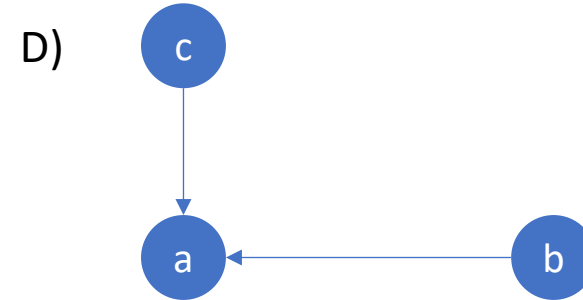
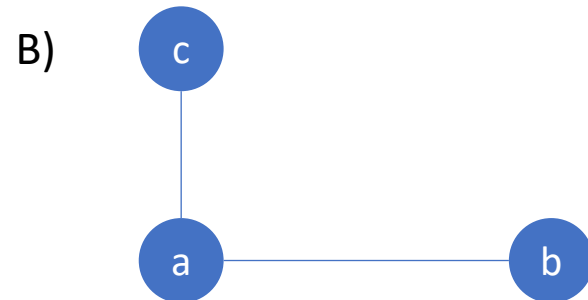


Übung

- Welcher Graph ist $(\{a,b,c\}, \{(a,c), (b,c)\})$?

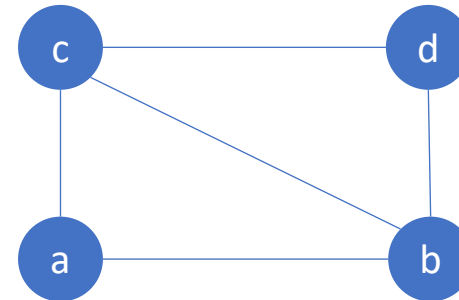


E)



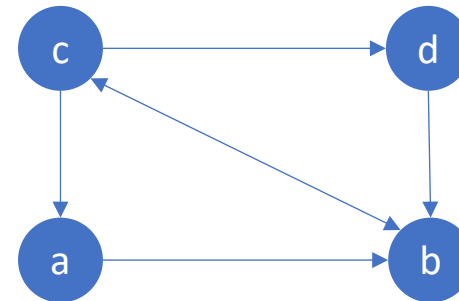
Übung

- Welche Art Graph ist dies?
 - A. kein korrekter Graph
 - B. ungerichteter Graph
 - C. gerichteter Graph
 - D. gewichteter Graph



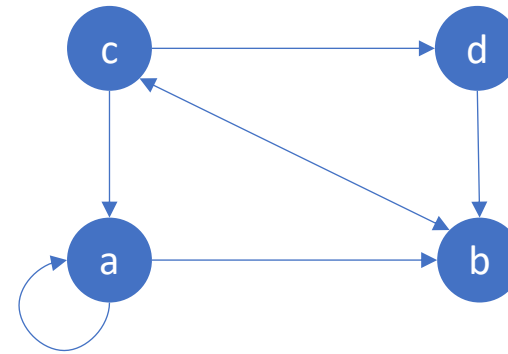
Übung

- Welche Art Graph ist dies?
 - A. kein korrekter Graph
 - B. ungerichteter Graph
 - C. gerichteter Graph
 - D. gewichteter Graph



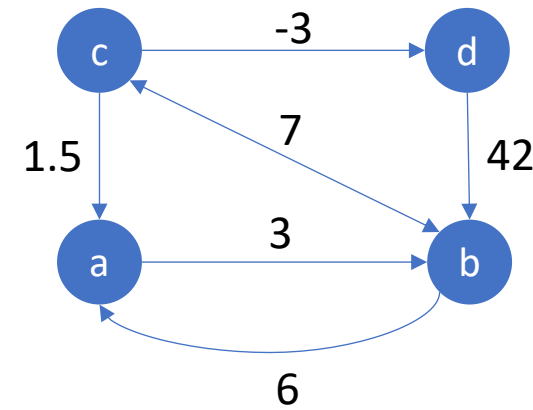
Übung

- Welche Art Graph ist dies?
 - A. kein korrekter Graph
 - B. ungerichteter Graph
 - C. gerichteter Graph
 - D. gewichteter Graph



Übung

- Welche Art Graph ist dies?
A. kein korrekter Graph
B. ungerichteter Graph
C. gerichteter Graph
D. gewichteter Graph





Realisierung von Graphen

- Übliche Datenstrukturen: Realisierungen unterscheiden sich im Platzbedarf bei der Speicherung und bei der Komplexität der Basisoperationen (Kante einfügen / löschen, Knoten einfügen / löschen)
 - dynamische Datenstruktur
 - Kanten- und Knotenlisten
 - Matrixdarstellung
- Direkte Realisierung als dynamische Datenstruktur
 - analog zu Bäumen
 - Knoten mit Zeiger-Liste
 - unüblich da komplexe Strukturen mit vielen Fehlermöglichkeiten (beim Speichern)

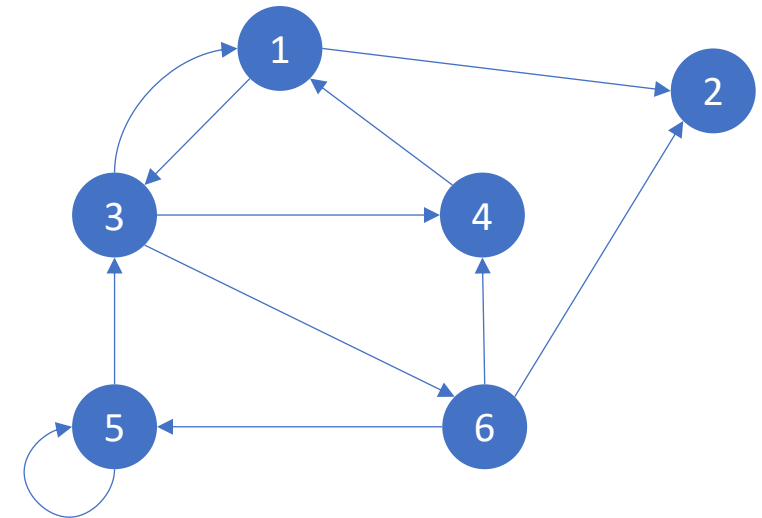


Kanten- und Knotenlisten

- einfache Realisierung bei durchnummerierten Knoten
- historisch erste verwendete Datenstruktur
- als Austauschformat geeignet
- Auflistung nach Knoten oder nach Kanten sortiert
- Knoten von 1 bis $|V|$ durchnummeriert.
- Kanten als Paare von Knotenzahlen dargestellt werden.
 - ebenfalls durchnummeriert
 - Knoten nicht explizit gespeichert, nur $|V|$ wird benötigt

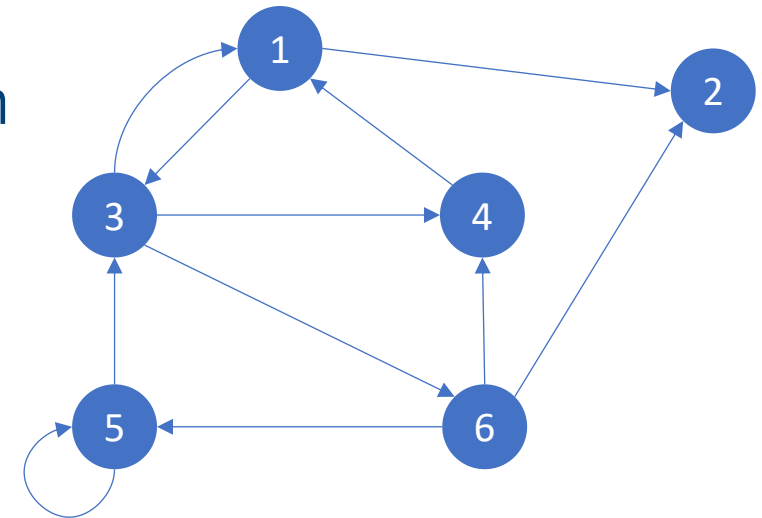
Kantenlisten

- 1. Element der Kantenliste: Knotenanzahl
- 2. Element der Kantenliste: Kantenanzahl
- Danach Liste der Kanten jeweils als 2 Zahlen dargestellt
- Kantenliste für G_g :
 - 6, 11, 1, 2, 1, 3, 3, 1, 4, 1, 3, 4, 3, 6, 5, 3, 5, 5, 6, 5, 6, 2, 6, 4



Knotenlisten

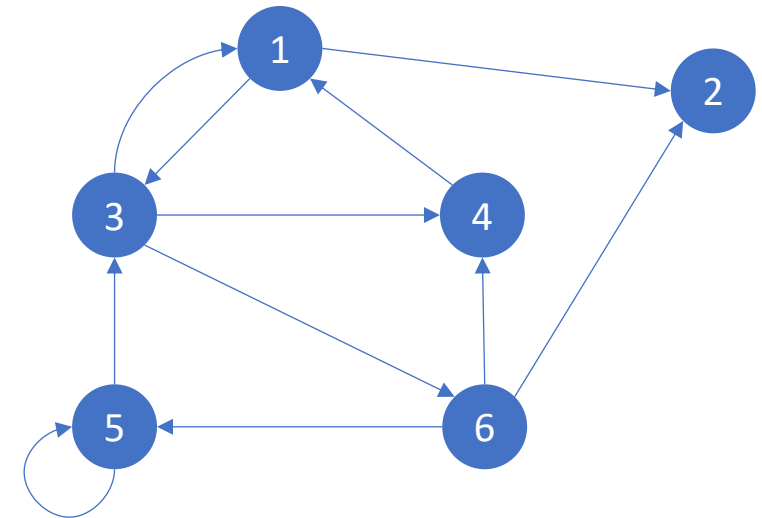
- 1. Element der Knotenliste: Knotenanzahl
- 2. Element der Knotenliste: Kantenanzahl
- Danach Liste der Kanteninformationen: Anzahl der von einem Knoten ausgehenden Kanten und die Liste der Zielknoten
- Dabei geht man die Knoten der Reihe nach durch: 1, 2, ...
- G_g als Knotenliste:
 - 6, 11, 2, 2, 3, 0, 3, 1, 4, 6, 1, 1, 2, 3, 5, 3, 2, 4, 5
- Teilfolge 2, 2, 3 bedeutet „Knoten 1 hat Ausgangsgrad 2 und herausgehende Kanten zu den Knoten 2 und 3“





Kanten- vs. Knotenlisten

- Knotenlisten benötigen weniger Speicher als Kantenlisten
- Kantenlisten: $2 + 2 \cdot |E|$
- Knotenlisten: $2 + |V| + |E|$
- Erinnerung: $|V|-1 \leq |E| \leq |V|^2$ für verbundene Graphen





Adjazenzmatrix

- Graphen $G=(V,E)$ ohne Schleifen und ohne Mehrfachkanten.
- Knoten und Kanten nummeriert
 - $V=\{v_1,\dots,v_n\}$ $E=\{e_1,\dots,e_m\}$
- Definition
 - Die **Adjazenzmatrix** eines gerichteten Graphen G ist eine $n \times n$ Matrix mit den Elementen
 - $a_{i,j} = 1$ falls $(v_i, v_j) \in E$,
 - $a_{i,j} = 0$ sonst.
 - Ungerichtete Graphen: symmetrisch
 - Keine Schleifen $\Rightarrow$ Hauptdiagonale 0



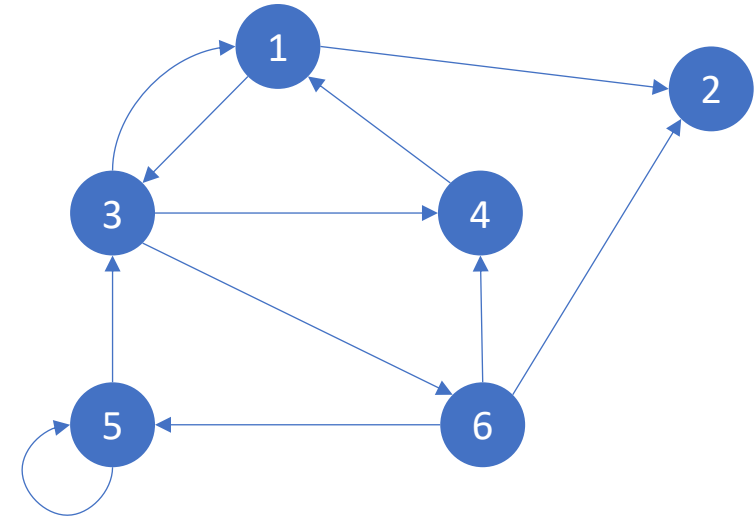
Operationen - Adjazenzmatrix

- Feststellen, ob eine Kante (a,b) existiert: $O(1)$
- Matrixmultiplikation A^k
 - Nur 0 oder 1: transitive Hülle
- Speicherbedarf $O(|V|^2)$
 - Meist viele 0-en
- Initialisierung Zeit $O(|V|^2)$
 - Wenn nur $O(|V|)$ Kanten, ineffizient.



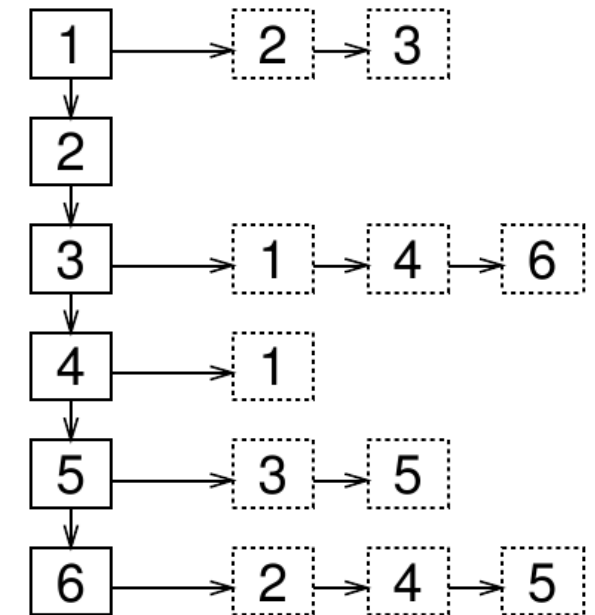
Beispiel - Adjazenzmatrix

$$\bullet G_g = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$



Adjazenzlisten

- Für jeden Knoten eine Liste von benachbarten Knoten .
- Platzbedarf $O(|V| + |E|)$
- vorteilhaft, wenn $|E| \ll |V|^2$
- Test, ob a und b direkt verbunden sind: $O(deg)$, wobei deg der maximale Grad des Graphen ist.





Transformation zwischen den Darstellungen

- Die vorgestellten Realisierungsvarianten sind äquivalent
 - jede Darstellung kann also in jede andere ohne Informationsverlust transformiert werden
 - Auslesen der einen Darstellung und anschließend das Erzeugen der jeweils anderen Darstellung
- Aufwand dieser Transformationen variiert von $O(n + m)$ bis $O(n^2)$, wobei im schlechtesten Fall $m = n^2$ gilt
- n^2 tritt immer auf, wenn eine Matrixdarstellung beteiligt ist



Komplexitätsbetrachtungen

- Seien $n = |V|$ und $m = |E|$.

Operation	Kantenliste	Knotenliste	Adjazenzmatrix	Adjazenzliste
Einfügen Kante	$O(1)$	$O(n+m)$	$O(1)$	$O(1) / O(n)$
Löschen Kante	$O(m)$	$O(n+m)$	$O(1)$	$O(n)$
Einfügen Knoten	$O(1)$	$O(1)$	$O(n^2)$	$O(1)$
Löschen Knoten	$O(m)$	$O(n+m)$	$O(n^2)$	$O(n+m)$

- Löschen eines Knotens erfordert Löschen aller zugehörigen Kanten.



Komplexitätsbetrachtungen

- Kantenlisten
 - Einfügen von Kanten (Anhängen zweier Zahlen) und von Knoten (Erhöhung der ersten Zahl um 1) besonders günstig
 - Löschen von Kanten: Zusammenschieben in einer Liste
 - Löschen von Knoten: Durchnummerierung der Knoten
- Knotenlisten
 - Einfügen von Knoten (Erhöhung der ersten Zahl und Anhängen einer 0) günstig
- Matrixdarstellung
 - Manipulieren von Kanten sehr effizient ausführbar
 - Aufwand bei Knoteneinfügung hängt von Realisierung ab – eventuell Kopieren der Matrix in eine größere Matrix
- Adjazenzlisten
 - unterschiedlicher Aufwand, je nachdem ob die Knotenliste als Feld (mit Direktzugriff) oder als verkettete Liste (mit sequenziellem Durchlauf) realisiert ist.



Implementierung in Java

- Node
 - Name (String)
 - Nachbarn als Nachbarschaftsliste `ArrayList`
 - Intern vergebene Knotennummer – klappt gut, wenn selten Knoten gelöscht werden.
- Edge
 - Kante: Ziel, Gewicht
 - Quelle: implizit bekannt.
- Zuordnung Knotenname $\rightarrow$ Knotennummer: `HashMap`



Implementierung Graph

```
public class Edge {  
    int dest, cost;  
    public Edge(int d, int c) {  
        dest = d; cost = c;  
    }  
}
```





Implementierung Graph

```
public class Node {  
    String name;  
    ArrayList<Edge> edges;  
  
    public Node(String n) {  
        name = n; edges = new ArrayList<>();  
    }  
  
    public List<Edge> getEdges() { return edges; }  
  
    public String toString() {  
        StringBuilder sb = new StringBuilder(name);  
        for (Edge k : edges) {  
            sb.append(" -- " + k.toString());  
        }  
        return sb.toString();  
    }  
}
```





Implementierung Graph



```
public class Graph {  
  
    private HashMap<String, Integer> labels;  
    private ArrayList<Node> nodes;  
  
    public Graph() {  
        labels = new HashMap<String, Integer>();  
        nodes = new ArrayList<Node>();  
    }  
  
    public void addNode(String label) throws NodeAlreadyDefinedException {  
        if (labels.containsKey(label))  
            throw new NodeAlreadyDefinedException();  
        nodes.add(new Node(label));  
        int idx = nodes.size() - 1;  
        labels.put(label, idx);  
    }  
    // ...  
}
```




Implementierung Graph



```
public int getNodeID(String label) {
    Integer i = labels.get(label);
    if (i == null)
        throw new NoSuchElementException();
    return i.intValue();
}

public void addEdge(String src, String dest, int cost) {
    Node node = nodes.get(getNodeID(src));
    node.getEdges().add(new Edge(getNodeID(dest), cost));
}

public Iterator<Edge> getEdges(int node) {
    return nodes.get(node).getEdges().iterator();
}
// ...
```



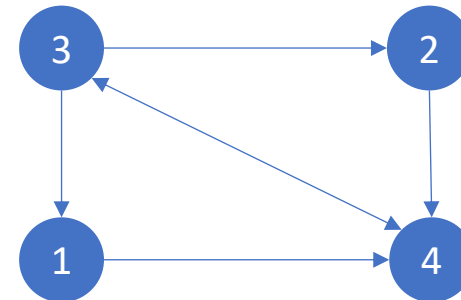
Implementierung Graph

```
public String toString() {  
    StringBuilder sb = new StringBuilder();  
    for (int i = 0; i < nodes.size(); i++) {  
        sb.append(i + ": ");  
        sb.append(nodes.get(i).toString() + "\n");  
    }  
    return sb.toString();  
}  
}
```



Übung

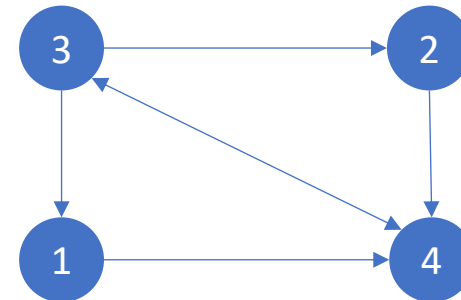
- Welche Kantenliste beschreibt den Graphen?
 - A. 4, 6, 1, 4, 1, 4, 3, 1, 2, 4, 1, 3
 - B. 4, 6, 1, 4, 1, 3, 2, 4, 2, 3, 3, 4
 - C. 4, 6, 1, 4, 3, 1, 4, 3, 2, 4, 3, 2, 3, 4
 - D. 1, 4, 3, 1, 4, 3, 2, 4, 3, 2, 3, 3



Übung

- Welche Knotenliste beschreibt den Graphen?

- A. 4, 6, 1, 4, 1, 4, 3, 1, 2, 4, 1, 3
- B. 4, 6, 1, 4, 1, 3, 2, 4, 2, 3, 3, 4
- C. 4, 6, 1, 4, 3, 1, 4, 3, 2, 4, 3, 2, 3, 4
- D. 1, 4, 3, 1, 4, 3, 2, 4, 3, 2, 3, 3



Übung

- Welche Adjazenzliste beschreibt den Graphen?
 - $A = \{ \{1\}, \{2,3\}, \{1, 4\}, \{3, 4\} \}$
 - $B = \{ \{4\}, \{3, 4\}, \{1, 2, 4\}, \{3\} \}$
 - $C = \{ \{1, 4\}, \{2, 3, 4\}, \{3, 1, 2, 4\}, \{4, 3\} \}$
 - $D = \{ (1, \{4\}), (2, \{3, 4\}), (3, \{1, 2, 4\}), (4, \{3\}) \}$
 - $E = \{ \{4\}, \{4, 3\}, \{4, 2, 1\}, \{3\} \}$

